Lecture 3: Ranking Models

Information Retrieval — Lecture Notes

BM25, Statistical Language Models, and Query Expansion

Avishek Anand
Information Retrieval Course

February 12, 2026

Abstract

These lecture notes accompany Lecture 3 of the Information Retrieval course, covering the three pillars of sparse ranking: **probabilistic relevance models (BM25)**, **statistical language models for ranking**, and **query expansion** (both classical and LLM-based). We develop each topic from first principles, provide worked examples with full arithmetic, and discuss practical considerations for production systems. The notes assume familiarity with inverted indexes and TF-IDF weighting from Lectures 1–2. By the end of this lecture, students will understand the scoring functions that underpin every modern retrieval pipeline and will be prepared for the transition to neural ranking in Lecture 4.

Contents

1 From Vector Space to Probabilistic Ranking

In Lecture 1, we introduced the *Vector Space Model* (VSM): documents and queries are represented as vectors in a high-dimensional term space, and relevance is measured by cosine similarity. In Lecture 2, we showed how inverted indexes make it feasible to compute these scores at scale, processing only documents that share at least one term with the query. The natural follow-up question is: *once we have candidate documents, how should we score them?*

TF-IDF cosine similarity is a reasonable starting point, but it has well-known limitations. Term frequency grows linearly, so a document mentioning "algorithm" 100 times scores 100× higher than one mentioning it once—an unrealistic model of relevance. This lecture introduces three families of scoring functions that address this and other shortcomings: **BM25**, **statistical language models**, and **query expansion methods**.

1.1 The Probability Ranking Principle

The theoretical foundation for probabilistic ranking was established by Robertson [**?**].

Theorem 1.1 (Probability Ranking Principle (PRP))**.** *If documents are ranked by decreasing probability of relevance given the query, i.e., $P(\text{relevant} \mid q, d)$, then the expected effectiveness of the retrieval system is maximized, under the assumption that the relevance of each document is independent of other documents.*

The PRP tells us *what* to optimize: rank by $P(\text{rel} \mid q, d)$. Every scoring function we study—TF-IDF, BM25, language models, and even neural rankers (Lecture 4)—can be understood as a different approximation of this ideal ranking.

Remark 1.1. The independence assumption is strong. In practice, a user who has already seen one relevant document gains less utility from a second document covering the same material. Extensions like Maximum Marginal Relevance (MMR) and diversity-aware ranking address this, but are beyond the scope of this lecture.

1.2 Limitations of TF-IDF

Recall the TF-IDF scoring function from Lecture 1:

$$\text{TF-IDF}(q, d) = \sum_{t \in q} \text{tf}(t, d) \cdot \log \frac{N}{\text{df}_t} \tag{1}$$

where $\text{tf}(t, d)$ is the raw term frequency, N is the total number of documents, and df_t is the document frequency of term t.

Three problems motivate the move to BM25:

1. **Linear TF growth.** Relevance increases with term frequency, but saturates quickly. The fifth occurrence of "algorithm" in a paper is informative; the fiftieth is likely noise. TF-IDF treats both increases linearly.

2. **No document length normalization.** A 10,000-word document naturally accumulates more term occurrences than a 100-word abstract. TF-IDF does not account for this.

3. **Heuristic IDF.** The standard $\log(N/\text{df}_t)$ is intuitive but not derived from a probabilistic model.

BM25 addresses all three issues within a principled probabilistic framework.

2 BM25: The Probabilistic Standard

BM25 (Best Matching 25) was developed by Robertson and colleagues as part of the Okapi project at City, University of London, and formalized in a series of TREC experiments in the 1990s. It remains the default ranking function in Elasticsearch, Solr, Lucene, and most production search engines today. The standard reference is Robertson and Zaragoza [**?**].

Historical Context

"BM25" stands for "Best Match, version 25." It was the 25th in a series of ranking functions explored by the Okapi team. Versions 1 through 24 are largely forgotten, but BM25 proved so effective that it has been the industry standard for over 30 years.

2.1 The Full Formula

Definition 2.1 (BM25)**.** Given a query $q = \{t_1, \ldots, t_n\}$ and a document d, the BM25 score is:

$$\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}}\right)} \tag{2}$$

where $f(t, d)$ is the frequency of term t in document d, $|d|$ is the length of d (in tokens), avgdl is the average document length in the collection, and k_1 and b are free parameters.

The formula decomposes into three interpretable components, which we examine in turn.

2.2 Component 1: IDF (Term Importance)

The BM25 IDF is derived from the Robertson–Spärck Jones weight [**?**]:

$$\mathrm{IDF}(t) = \log \frac{N - \mathrm{df}_t + 0.5}{\mathrm{df}_t + 0.5} \tag{3}$$

This differs from the standard TF-IDF IDF in two ways:

- The $+0.5$ smoothing terms prevent division by zero and provide a continuity correction.
- The numerator subtracts df_t, which means that for very common terms ($\mathrm{df}_t > N/2$), the IDF becomes *negative*. This actively penalizes matches on extremely frequent terms—a desirable property that standard IDF lacks.

Example 2.1 (IDF values)**.** In a collection of $N = 100{,}000$ documents:

Term	df_t	IDF	**Interpretation**
"quantum"	100	6.91	Very discriminative
"algorithm"	5,000	2.94	Moderately discriminative
"system"	50,000	-0.41	Not discriminative (negative)
"the"	99,000	-6.90	Actively harmful

Rare terms like "quantum" carry most of the scoring weight. A match on "quantum" is worth roughly $6.91/2.94 \approx 2.3\times$ a match on "algorithm" and infinitely more than a match on "the."

2.3 Component 2: Saturating Term Frequency

The TF component of BM25 is:

$$\frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1} \tag{4}$$

This is a monotonically increasing function of $f(t,d)$ that asymptotically approaches $k_1 + 1$. The parameter k_1 controls the rate of saturation:

- $k_1 \to 0$: The function collapses to a constant. Term frequency is ignored entirely, yielding Boolean matching (present/absent).
- $k_1 \to \infty$: The function becomes linear in $f(t,d)$, recovering TF-IDF behavior.
- $k_1 = 1.2$ (standard): Moderate saturation. The first few occurrences matter most; additional occurrences have diminishing impact.

Example 2.2 (Saturation values for $k_1 = 1.2$)**.**

$f(t,d)$	TF component	Fraction of maximum
1	1.00	45%
2	1.38	63%
5	1.77	81%
10	1.96	89%
50	2.15	98%
100	2.18	99%

The first occurrence accounts for 45% of the maximum contribution. By 10 occurrences, we are already at 89%. This is the "saturation" property that TF-IDF lacks.

2.4 Component 3: Document Length Normalization

In the full BM25 formula (Equation **??**), the denominator contains the length normalization term:

$$1 - b + b \cdot \frac{|d|}{\text{avgdl}} \tag{5}$$

This multiplicatively inflates the effective k_1 for long documents and deflates it for short documents:

- $b = 0$: No length normalization. All documents treated equally regardless of length.
- $b = 1$: Full normalization. A document twice the average length has its TF contribution roughly halved.
- $b = 0.75$ (standard): Partial normalization. Long documents are penalized, but not as aggressively as $b = 1$.

The intuition is that long documents accumulate more term occurrences simply by virtue of having more text, not necessarily because they are more relevant. The parameter b controls how strongly we penalize this.

Remark 2.1 (When to adjust b)**.** For collections with highly variable document lengths (e.g., web pages ranging from 10 to 10,000 words), a higher b (0.8–0.9) is appropriate. For collections with uniform lengths (e.g., abstracts, tweets), a lower b (0.3–0.5) may work better, since length variation carries less signal about relevance.

2.5 Worked Example: End-to-End BM25 Scoring

We now walk through a complete BM25 computation to build intuition for how the components interact.

Example 2.3 (Full BM25 scoring)**. Setup.** Query: "quantum algorithm." Collection: $N = 100{,}000$ documents, avgdl = 500 words. Parameters: $k_1 = 1.2$, $b = 0.75$.

Two candidate documents:

- Doc A: 300 words. "quantum" appears 5 times, "algorithm" appears 3 times.
- Doc B: 800 words. "quantum" appears 8 times, "algorithm" appears 1 time.

Collection statistics: df(quantum) = 200, df(algorithm) = 5,000.

Step 1: IDF.

$$\text{IDF(quantum)} = \log \frac{100{,}000 - 200 + 0.5}{200 + 0.5} = \log(497.8) = 6.21$$
$$\text{IDF(algorithm)} = \log \frac{100{,}000 - 5{,}000 + 0.5}{5{,}000 + 0.5} = \log(19.0) = 2.94$$

Step 2: Length normalization.

$$\text{Doc A}: \quad L_A = 1 - 0.75 + 0.75 \cdot \frac{300}{500} = 0.25 + 0.45 = 0.70$$
$$\text{Doc B}: \quad L_B = 1 - 0.75 + 0.75 \cdot \frac{800}{500} = 0.25 + 1.20 = 1.45$$

Step 3: TF components.
For Doc A ($L_A = 0.70$):

$$\text{TF}_A(\text{quantum}) = \frac{5 \times 2.2}{5 + 1.2 \times 0.70} = \frac{11.0}{5.84} = 1.88$$
$$\text{TF}_A(\text{algorithm}) = \frac{3 \times 2.2}{3 + 1.2 \times 0.70} = \frac{6.6}{3.84} = 1.72$$

For Doc B ($L_B = 1.45$):

$$\text{TF}_B(\text{quantum}) = \frac{8 \times 2.2}{8 + 1.2 \times 1.45} = \frac{17.6}{9.74} = 1.81$$
$$\text{TF}_B(\text{algorithm}) = \frac{1 \times 2.2}{1 + 1.2 \times 1.45} = \frac{2.2}{2.74} = 0.80$$

Step 4: Final scores.

$$\text{BM25}(q, A) = 6.21 \times 1.88 + 2.94 \times 1.72 = 11.67 + 5.06 = \mathbf{16.73}$$
$$\text{BM25}(q, B) = 6.21 \times 1.81 + 2.94 \times 0.80 = 11.24 + 2.35 = \mathbf{13.59}$$

Result: Doc A (16.73) > Doc B (13.59). Doc A wins despite having fewer occurrences of "quantum" (5 vs. 8). Two factors explain this: (1) Doc A is shorter than average ($L_A = 0.70 < 1$), so its term frequencies are boosted; Doc B is longer than average ($L_B = 1.45 > 1$), so its frequencies are dampened. (2) Doc A has 3 occurrences of "algorithm" while Doc B has only 1, and both query terms contribute to the score.

Key Takeaway

BM25 rewards documents that cover *all* query terms (not just one), that are shorter than average (unless penalized by b), and that have moderate term frequencies (due to saturation).

2.6 BM25F: Fielded Documents

Real documents are not flat bags of words. A web page has a title, body, URL, anchor text, and possibly meta-tags. A match in the title should carry more weight than a match in the body. BM25F [**?**] extends BM25 to handle multiple fields.

Definition 2.2 (BM25F)**.** Replace the raw term frequency $f(t, d)$ with a weighted combination across fields:

$$\tilde{f}(t, d) = \sum_{z \in \text{fields}} w_z \cdot \frac{f(t, d_z)}{1 - b_z + b_z \cdot \frac{|d_z|}{\text{avgdl}_z}} \tag{6}$$

where w_z is the importance weight for field z, b_z is the per-field length normalization parameter, and $|d_z|$ and avgdl_z are the length and average length of field z, respectively. The combined $\tilde{f}$ is then used in the standard BM25 formula in place of $f(t, d)$.

Typical field weights in web search: title $w = 5$–10, body $w = 1$, URL $w = 2$–3, anchor text $w = 3$–5. BM25F is the default ranking function in Elasticsearch and Solr, and is used in Bing's ranking stack.

2.7 Parameter Tuning

The defaults $k_1 = 1.2$ and $b = 0.75$ work well across a wide range of collections, but optimal values depend on the corpus:

Collection type	k_1 **guidance**	b **guidance**
Short documents (tweets, titles)	Lower (0.8–1.2)	Lower (0.3–0.5)
Standard documents (news, papers)	Default (1.2)	Default (0.75)
Verbose documents (web, books)	Higher (1.5–2.0)	Higher (0.8–0.9)

In Lecture 6 (Learning to Rank), we will see how these parameters—along with many others—can be tuned automatically using labeled relevance data.

3 Statistical Language Models for Ranking

BM25 asks: "How well does this document match the query?" Language models for IR ask a fundamentally different question: "How likely is it that this document would *generate* the query?" This generative perspective, introduced to IR by Ponte and Croft [**?**] and refined by many others, provides an elegant alternative with a solid theoretical foundation.

3.1 The Query Likelihood Model

Definition 3.1 (Query Likelihood Model)**.** Each document d defines a multinomial probability distribution $P(\cdot \mid d)$ over the vocabulary. A query $q = (w_1, w_2, \ldots, w_n)$ is modeled as a sequence of independent draws from this distribution. Documents are ranked by the probability they would generate q:

$$P(q \mid d) = \prod_{w \in q} P(w \mid d)^{\text{count}(w,q)} \tag{7}$$

The **generative story** is as follows: (1) pick a document d at random; (2) sample words from d's language model; (3) if the sampled words match the query, the document is relevant. Documents whose language models assign higher probability to the query words are ranked higher.

The maximum likelihood estimate (MLE) of the language model is:

$$P_{\text{ML}}(w \mid d) = \frac{\text{count}(w, d)}{|d|} \tag{8}$$

This has a fatal flaw.

3.2 The Zero Probability Problem

Example 3.1 (Zero probability kills relevant documents)**.** Query: "machine learning algorithms." Document: "This paper presents a novel neural network approach to supervised machine learning" (15 words).

Term	**count**(w, d)	$P_{\text{ML}}(w \mid d)$	**Contribution**
machine	1	0.067	✓
learning	1	0.067	✓
algorithms	0	0	× ZERO
		$P(q \mid d)$	$0.067 \times 0.067 \times 0 = \mathbf{0}$

The document is clearly relevant—it discusses machine learning and uses closely related terminology—but because "algorithms" does not appear literally, the entire score is zero. The single missing term kills the product.

This is not an edge case. In any realistic collection, most documents will lack at least one query term. Without smoothing, the query likelihood model assigns zero to nearly every document.

3.3 Smoothing: Mixing Document and Collection Evidence

The solution is to *smooth* the document language model by mixing it with a collection-wide background model:

$$P(w \mid C) = \frac{\sum_{d' \in C} \text{count}(w, d')}{\sum_{d' \in C} |d'|} \tag{9}$$

This collection language model represents the "typical" distribution of words across the entire corpus. Smoothing ensures that every word has a non-zero probability, even if it does not appear in the document.

There are two standard smoothing methods, each with distinct properties.

3.3.1 Jelinek-Mercer Smoothing

Definition 3.2 (Jelinek-Mercer (JM) Smoothing)**.**

$$P(w \mid d) = (1 - \lambda)\, P_{\text{ML}}(w \mid d) + \lambda\, P(w \mid C) \tag{10}$$

where $\lambda \in [0, 1]$ is a fixed interpolation parameter, typically 0.1–0.3.

JM smoothing linearly interpolates between the document model and the collection model. When $\lambda = 0$, we recover the unsmoothed MLE (with its zero-probability problem). When

$\lambda = 1$, we ignore the document entirely and rank by the background model, which assigns the same score to every document.

The key property of JM smoothing is that λ is **fixed for all documents**: a 100-word abstract and a 10,000-word article receive the same amount of smoothing. This can be problematic because we should arguably trust the MLE more for longer documents (which provide more data) and rely more on the collection model for shorter documents (which provide less data).

3.3.2 Dirichlet Smoothing

Definition 3.3 (Dirichlet Smoothing)**.**

$$P(w \mid d) = \frac{\text{count}(w, d) + \mu \cdot P(w \mid C)}{|d| + \mu} \tag{11}$$

where $\mu > 0$ is the smoothing parameter, typically 1,000–2,500.

Dirichlet smoothing has a Bayesian interpretation: it treats $P(w \mid C)$ as a Dirichlet prior on the document's multinomial distribution, with μ controlling the strength of the prior. Equivalently, we add μ "pseudo-counts" distributed according to the collection language model.

The critical advantage over JM smoothing is that Dirichlet smoothing is **length-adaptive**:

- For a **short document** ($|d| \ll \mu$), the denominator is dominated by μ, so the smoothed probability is close to the collection model. We trust the document's own statistics less.
- For a **long document** ($|d| \gg \mu$), the denominator is dominated by $|d|$, so the smoothed probability is close to the MLE. We trust the document's own statistics more.

This automatic adaptation to document length is analogous to BM25's b parameter, which controls length normalization. In both cases, the goal is to avoid over-crediting long documents for term frequencies that arise from length rather than relevance.

Example 3.2 (Dirichlet length adaptation)**.** Term "algorithm" with $P(w \mid C) = 0.001$ and $\mu = 2{,}000$.

Document	$\text{count}(w, d)$	$\lvert d \rvert$	$P(w \mid d)$
Short (100 words)	2	100	$\frac{2+2}{2{,}100} = 0.0019$
Long (2,000 words)	2	2,000	$\frac{2+2}{4{,}000} = 0.0010$

Same raw count (2), but the short document gets a higher smoothed probability. The rationale: two occurrences in 100 words is more informative than two occurrences in 2,000 words.

3.4 Comparing Smoothing Methods

Aspect	**Jelinek-Mercer**	**Dirichlet**
Formula	$(1-\lambda)P_{\text{ML}}(w\|d) + \lambda P(w\|C)$	$\frac{\text{count}(w,d)+\mu P(w\|C)}{\|d\|+\mu}$
Parameter	λ (0.1–0.3)	μ (1,000–2,500)
Length adaptation	None (fixed λ)	Automatic
Interpretation	Linear interpolation	Bayesian pseudo-counts
Best for	Uniform-length collections	Variable-length collections
Standard in	—	Indri, Galago, Terrier

In practice, Dirichlet smoothing is generally preferred for most collections due to its length-adaptive property. Zhai and Lafferty [**?**] conducted an extensive empirical study comparing smoothing methods and found Dirichlet smoothing to be the most robust across different collections and query types.

3.5 BM25 vs. Language Models

Despite their different philosophical foundations—BM25 is a "matching" model while the query likelihood model is a "generative" model—the two approaches produce surprisingly similar rankings when well-tuned. The deep connection is that both methods perform three fundamental operations:

1. **TF weighting:** BM25 uses explicit saturation (k_1); LMs achieve saturation implicitly through the count-to-probability conversion.
2. **IDF weighting:** BM25 includes IDF explicitly; in the LM framework, rare words naturally get higher probability ratios because $P_{\text{ML}}(w \mid d)$ is high relative to $P(w \mid C)$.
3. **Length normalization:** BM25 uses the b parameter; Dirichlet smoothing adapts automatically based on $|d|$ relative to μ.

Production Note

In production, BM25 is more commonly used than language models as the first-stage ranker, primarily because its parameters (k_1, b) are better understood and its defaults are more robust. Language models see more use in specialized applications (e.g., passage retrieval, cross-lingual IR) and as the theoretical foundation for query expansion via RM3.

4 Query Expansion and Pseudo-Relevance Feedback

BM25 and language models share a fundamental limitation: they can only match *exact terms*. If a user searches for "heart attack" but the most relevant document uses "myocardial infarction," no amount of TF weighting or smoothing will help—the BM25 score is zero because the terms do not overlap. This is the **vocabulary mismatch problem**, and it is perhaps the most important open problem in sparse retrieval.

Query expansion addresses vocabulary mismatch by adding related terms to the original query before retrieval. There are three generations of query expansion techniques: explicit relevance feedback (impractical), pseudo-relevance feedback (classical), and LLM-based expansion (modern).

4.1 Explicit Relevance Feedback

In explicit relevance feedback, the user marks retrieved documents as relevant or non-relevant, and the system uses this information to reformulate the query. The most famous method is the Rocchio algorithm [**?**], which modifies the query vector in TF-IDF space:

$$\vec{q}_{\text{new}} = \alpha\, \vec{q}_{\text{orig}} + \beta \cdot \frac{1}{|R|} \sum_{d \in R} \vec{d} \; - \; \gamma \cdot \frac{1}{|NR|} \sum_{d \in NR} \vec{d} \tag{12}$$

where R is the set of relevant documents, NR is the set of non-relevant documents, and α, β, γ control the balance between the original query, positive feedback, and negative feedback.

The geometric intuition is simple: move the query vector toward the centroid of relevant documents and away from non-relevant ones. However, explicit feedback is impractical in most settings: users are unwilling to provide judgments, and the system needs good initial results to show users in the first place (a chicken-and-egg problem).

4.2 Pseudo-Relevance Feedback (PRF)

Pseudo-relevance feedback (PRF) sidesteps the need for user interaction by making a bold assumption: **the top-k documents retrieved by the initial query are relevant.** Under this assumption, terms that are prominent in the top-k but not in the original query are likely related to the information need and should be added.

The PRF process has four steps:

1. Retrieve initial results with the original query (e.g., BM25 top-1000).
2. Take the top-k documents as "pseudo-relevant" (k typically 3–10).
3. Extract and weight terms from these documents.
4. Expand the query with the highest-weighted new terms and re-retrieve.

4.2.1 RM3: The Modern Standard for Classical PRF

RM3 (Relevance Model 3) is the most widely used PRF method today. It was introduced by Lavrenko and Croft [**?**] and provides a principled, probabilistic framework for expansion within the language modeling paradigm.

Definition 4.1 (RM3)**.** Given a set of pseudo-relevant documents R (the top-k from initial retrieval), the relevance model estimates the probability of each word w in the "relevance language":

$$P(w \mid R) \propto \sum_{d \in R} P(w \mid d) \cdot P(d \mid q) \tag{13}$$

where $P(w \mid d)$ is the (smoothed) document language model and $P(d \mid q)$ is the initial retrieval score, normalized to a probability.

The expanded query model is then formed by interpolating the original query model with the relevance model:

$$P(w \mid Q_{\text{new}}) = (1 - \lambda)\, P(w \mid Q_{\text{orig}}) + \lambda\, P(w \mid R) \tag{14}$$

where $\lambda \in [0, 1]$ balances the original query against the expansion terms.

In practice, only the top `fb_terms` words by $P(w \mid R)$ are retained (the rest are discarded to reduce noise), and the expanded query is used for a second round of retrieval.

Remark 4.1 (RM3 vs. RM1)**.** RM1 uses the relevance model directly as the query (Equation **??**). RM3 adds the interpolation step (Equation **??**), which preserves the original query terms and prevents *query drift*—the phenomenon where expansion terms dominate and shift the query away from its original intent. RM3 is almost universally preferred in practice.

Key parameters:

`fb_docs` Number of pseudo-relevant documents (3–10; standard: 10). Too few may miss relevant terms; too many risk incorporating non-relevant documents.

`fb_terms` Number of expansion terms to add (10–20; standard: 10). Too few limits the benefit; too many adds noise.

λ Interpolation weight (0.5–0.8). Lower values trust the original query more; higher values trust the expansion more.

Example 4.1 (RM3 expansion)**.** Original query: "machine learning." The top-3 BM25 results contain terms related to neural networks, supervised learning, and AI. RM3 extracts: "neural" ($P = 0.15$), "deep" ($P = 0.12$), "supervised" ($P = 0.10$), "algorithms" ($P = 0.09$), "AI" ($P = 0.08$). The expanded query (with $\lambda = 0.6$) becomes: "machine learning neural deep supervised algorithms AI." The second retrieval round now finds documents about machine learning that use these related terms even if they don't contain "machine learning" literally.

4.2.2 Query Drift

The Achilles' heel of PRF is **query drift**: when the top-k documents happen to be off-topic, the expansion terms are wrong, and the re-retrieved results are even worse. This is a positive feedback loop—bad initial results lead to bad expansion, which leads to worse results.

Common Pitfall

Query drift is most severe for ambiguous queries. Example: the query "python programming" returns an article about Python snakes as the top result. RM3 extracts "reptile," "snake," "habitat," "venom." The re-retrieved results are dominated by herpetology, and the programming intent is lost entirely.

Strategies to mitigate query drift include: keeping λ moderate (preserving the original query), using fewer `fb_docs`, filtering expansion terms by IDF, and—as we discuss next—using LLMs to generate expansion terms directly.

4.3 LLM-Based Query Expansion

Starting around 2022, researchers discovered that large language models (LLMs) can generate effective expansion terms without any retrieval at all, using their *parametric knowledge* (information encoded in the model's weights during pre-training). This eliminates the dependency on the quality of initial results and avoids the query drift problem inherent in PRF.

We describe four strategies, each addressing vocabulary mismatch from a different angle.

4.3.1 Strategy 1: Synonym and Paraphrase Generation

The simplest approach is to prompt an LLM to generate synonyms, related terms, or paraphrases of the query.

Prompt template:

```
Given the search query: ``{query}''
Generate 5 alternative phrasings and domain-specific synonyms that a relevant
document might use. Return only the terms, one per line.
```

For the query "heart attack symptoms," the LLM might generate: "myocardial infarction signs," "cardiac arrest warning indicators," "chest pain emergency symptoms," "coronary event presentation," "acute heart failure signs." The expanded BM25 query now includes medical terminology that bridges the vocabulary gap between lay users and clinical documents.

This approach is particularly effective in domains with strong vocabulary divergence (medical, legal, scientific) and for abbreviation expansion ("ML" $\rightarrow$ "machine learning").

4.3.2 Strategy 2: Hypothetical Document Embeddings (HyDE)

Gao et al. [**?**] introduced a more creative approach: instead of generating individual terms, ask the LLM to write an entire hypothetical document that would be a perfect answer to the query.

Prompt template:

```
Write a short paragraph that would be a perfect answer to the question:
``{query}''
```

For the query "What causes ocean acidification?", the LLM generates a paragraph mentioning "carbon dioxide," "carbonic acid," "pH," "calcium carbonate shells," "corals," "mollusks"—all terms that a relevant scientific document would contain but that a user might not include in their query.

The hypothetical document can be used in two ways:

1. **Sparse expansion:** Extract key terms from the generated text and add them to the BM25 query.
2. **Dense expansion:** Encode the generated text with a dense encoder and use the resulting embedding as the query vector for approximate nearest neighbor search (see Lecture 5).

The second usage is particularly powerful because it sidesteps the need for relevance labels entirely—the LLM-generated document serves as a "zero-shot" query representation.

4.3.3 Strategy 3: Chain-of-Thought Query Decomposition

Complex, multi-faceted queries often perform poorly because a single retrieval pass cannot capture all aspects simultaneously. LLMs can decompose such queries into simpler sub-queries.

Prompt template:

```
I need to answer: ``{query}''
Break this into 3--4 simpler search queries that together would cover the
full answer.
```

For the query "How did the 2008 financial crisis affect European housing markets?", the LLM decomposes into: (1) "2008 financial crisis causes subprime mortgage," (2) "European housing prices 2008 2009 decline," (3) "EU bank bailouts housing market impact," (4) "European mortgage lending regulations post-crisis." Each sub-query is run through BM25 independently, and the results are merged (e.g., via Reciprocal Rank Fusion, covered in Lecture 5).

This strategy is related to the *multi-hop reasoning* literature in question answering, where complex questions require chaining multiple retrieval steps. It is especially valuable for RAG (Retrieval-Augmented Generation) systems [**?**].

4.3.4 Strategy 4: Query2Doc

Wang et al. [**?**] proposed a particularly simple integration: generate a pseudo-document with an LLM and **prepend it to the original query** before running standard BM25. The concatenated text serves as the query, with the original terms providing precision and the generated text providing recall.

Example: Original query: "dark matter detection methods." The LLM generates a paragraph mentioning "cryogenic detectors," "gamma-ray observations," "Large Hadron Collider," "direct detection," "indirect detection." The concatenation is fed to BM25 as a single (long) query.

Advantages: Drops into any existing BM25 pipeline with no architectural changes. The LLM runs once per query (offline), so latency at retrieval time is unchanged.

Risks: LLM hallucinations introduce irrelevant terms as noise. The expanded "query" is much longer, which changes BM25's behavior (long queries reduce the discriminative power of individual terms). Weighting the original query more heavily than the generated text can mitigate this.

4.4 Classical vs. LLM Expansion: A Comparison

Dimension	Classical (RM3)	LLM-Based
Knowledge source	Retrieved documents	Parametric (pre-training data)
Failure mode	Query drift (bad top-k)	Hallucination
Latency overhead	Minimal (statistics only)	LLM inference (100ms–1s)
Cost	Free	API cost or GPU
Domain adaptation	Automatic (uses local corpus)	Requires prompting or fine-tuning
Cold-start	Fails (needs populated index)	Works (no index required)
Interpretability	Terms from real documents	Generated terms

Key Takeaway

Classical and LLM-based expansion are **complementary**: RM3 leverages local corpus statistics while LLMs leverage broad world knowledge. Current best practice (2024+) is to combine both: use LLM expansion to improve the initial retrieval, then apply RM3 on the (now better) top-k results. Empirically, this combination outperforms either method alone [**?**, **?**].

4.5 When Does Query Expansion Help vs. Hurt?

Query expansion is not universally beneficial. Understanding when it helps and when it hurts is essential for system design.

Helps when:

- Initial results are mostly on-topic (PRF assumption holds).
- Vocabulary mismatch is the primary source of recall failure.
- Queries are broad and informational ("climate change" → adds "warming," "CO2," "emissions").

Hurts when:

- Initial results are off-topic (query drift amplifies errors).
- Queries are ambiguous ("jaguar" → adds "car," "speed" when the user meant the animal).
- Queries are highly specific or navigational ("OpenAI API documentation" needs exact match, not expansion).

In production systems, query expansion is often applied *selectively*: the system predicts whether expansion is likely to help for a given query (e.g., based on the coherence of the top-k results) and only applies it when the prediction is positive.

5 Putting It Together: The Sparse Ranking Stack

The methods in this lecture form the sparse ranking component of the modern retrieval pipeline. Before moving to neural methods in Lecture 4, it is worth reflecting on how BM25, language models, and query expansion compose into a coherent system.

5.1 The Standard Pipeline

A typical sparse retrieval pipeline consists of three stages:

1. **First-stage retrieval:** BM25 (or Dirichlet LM) over the inverted index retrieves the top-1,000 candidate documents. This is the recall-oriented stage: the goal is to include as many relevant documents as possible, even at the cost of some irrelevant ones.
2. **Query expansion (optional):** RM3 or LLM expansion improves recall by adding related terms and re-retrieving. This is especially valuable when vocabulary mismatch is a concern.
3. **Re-ranking (Lecture 4):** A neural cross-encoder re-ranks the top-100 candidates for precision. This is the accuracy-oriented stage.

The sparse stages (1 and 2) set the *recall ceiling* for the entire pipeline: if a relevant document is not retrieved by BM25, no downstream re-ranker can recover it. This is why BM25 remains critical even in the era of neural ranking.

5.2 Connections to Future Lectures

Every method introduced today reappears in later lectures:

- **Lecture 4 (Embeddings & Re-rankers):** BERT cross-encoders re-rank BM25's top-k candidates. The quality of BM25's candidate set directly determines the ceiling for neural re-ranking.
- **Lecture 5 (Dense Retrieval):** Bi-encoders replace or complement BM25 as the first-stage retriever. HyDE connects LLM expansion to dense retrieval. Hybrid retrieval combines BM25 and dense signals.
- **Lecture 6 (Learning to Rank):** BM25 scores, LM scores, and query expansion features all become inputs to a learned ranking function (LambdaMART). The parameters k_1, b, μ, λ, `fb_docs`, and `fb_terms` are all tunable via supervised learning.

5.3 Fundamental Limitations of Sparse Methods

Despite their effectiveness and efficiency, sparse methods share several fundamental limitations:

1. **Bag-of-words assumption.** Word order is ignored: "dog bites man" and "man bites dog" receive identical scores.
2. **Exact term matching.** Even with query expansion, sparse methods ultimately rely on term overlap. "Car" and "automobile" remain completely different tokens.
3. **No semantic understanding.** Negation ("not relevant" vs. "relevant"), context ("bank" as financial institution vs. riverside), and intent (informational vs. navigational) are invisible to term-based scoring.

These limitations motivate the transition to dense, neural representations in Lecture 4.

Key Takeaway

BM25 is not going away. It is the **foundation layer** of every modern retrieval system—as the first-stage retriever, as a feature in learned rankers, and as the baseline that every neural method must beat. Understanding BM25 deeply is essential for anyone working in information retrieval.

6 Hands-On: PyTerrier Experiments

This section provides code examples for implementing and comparing the ranking methods discussed above using PyTerrier [**?**].

6.1 Setup and BM25 Baseline

```python
import pyterrier as pt
if not pt.started():
    pt.init()

# Load the Vaswani test collection (small, fast for teaching)
dataset = pt.get_dataset('irds:vaswani')
index = pt.IndexFactory.of("./vaswani_index")
topics = dataset.get_topics().head(10)
qrels = dataset.get_qrels()

# Default BM25 (k1=1.2, b=0.75)
```

```
bm25 = pt.BatchRetrieve(index, wmodel="BM25")
results = bm25.transform(topics)
```

6.2 BM25 Parameter Experiments

```
# BM25 variants with different parameters
bm25_default = pt.BatchRetrieve(index, wmodel="BM25")
bm25_high_k1 = pt.BatchRetrieve(index, wmodel="BM25",
                                controls={"bm25.k1": "2.0"})
bm25_low_b   = pt.BatchRetrieve(index, wmodel="BM25",
                                controls={"bm25.b": "0.3"})

# Dirichlet LM with different mu values
lm_1000 = pt.BatchRetrieve(index, wmodel="DirichletLM",
                           controls={"c": "1000"})
lm_2500 = pt.BatchRetrieve(index, wmodel="DirichletLM",
                           controls={"c": "2500"})

# Compare all systems
pt.Experiment(
    [bm25_default, bm25_high_k1, bm25_low_b, lm_1000, lm_2500],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "P_10"],
    names=["BM25 default", "BM25 k1=2.0", "BM25 b=0.3",
           "LM mu=1000", "LM mu=2500"]
)
```

6.3 RM3 Query Expansion

```
# RM3 pipeline: retrieve -> expand -> re-retrieve
rm3 = pt.rewrite.RM3(index)  # Default: fb_docs=10, fb_terms=10
pipeline_rm3 = bm25 >> rm3 >> bm25

# Compare BM25 vs. BM25+RM3
pt.Experiment(
    [bm25, pipeline_rm3],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "recall_100"],
    names=["BM25", "BM25 + RM3"]
)

# Inspect the expanded query
expanded = (bm25 >> rm3).transform(topics)
print(expanded[expanded['qid']=='1']['query'].values[0])
```

6.4 LLM-Based Expansion

```
from openai import OpenAI
client = OpenAI()

def llm_expand(query, n_terms=5):
    """Expand query using LLM synonym generation."""
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content":
            f"Generate {n_terms} search terms related to: "
            f"'{query}'. Return only the terms, comma-separated."}]
    )
```

```
    return f"{query} {resp.choices[0].message.content}"

# Expand all topics
topics_llm = topics.copy()
topics_llm['query'] = topics_llm['query'].apply(llm_expand)

# BM25 on LLM-expanded queries
results_llm = bm25.transform(topics_llm)
```

7 Summary

This lecture introduced three families of sparse ranking methods:

1. **BM25** provides a principled probabilistic scoring function with three components: IDF (term importance), saturating TF (diminishing returns on frequency), and length normalization (penalizing long documents). The standard parameters $k_1 = 1.2$ and $b = 0.75$ work well for most collections. BM25F extends the model to fielded documents.

2. **Statistical language models** offer an alternative generative framework: rank by $P(q \mid d)$, the probability that the document's language model would generate the query. Smoothing is essential to avoid zero probabilities; Dirichlet smoothing is preferred for its length-adaptive property.

3. **Query expansion** addresses the vocabulary mismatch problem. Classical methods (RM3) extract terms from pseudo-relevant documents; modern methods use LLMs to generate expansion terms from parametric knowledge (synonym generation, HyDE, query decomposition, Query2Doc). The two approaches are complementary.

All three families share fundamental limitations: bag-of-words representation, exact term matching, and no semantic understanding. These limitations motivate the transition to dense, neural representations in Lecture 4.

References

[1] S.E. Robertson. The probability ranking principle in IR. *Journal of Documentation*, 33(4):294–304, 1977.

[2] S.E. Robertson and K. Spärck Jones. Relevance weighting of search terms. *Journal of the American Society for Information Science*, 27(3):129–146, 1976.

[3] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[4] S. Robertson, H. Zaragoza, and M. Taylor. Simple BM25 extension to multiple weighted fields. In *Proceedings of CIKM*, pages 42–49, 2004.

[5] J.M. Ponte and W.B. Croft. A language modeling approach to information retrieval. In *Proceedings of SIGIR*, pages 275–281, 1998.

[6] C. Zhai and J. Lafferty. A study of smoothing methods for language models applied to information retrieval. *ACM Transactions on Information Systems*, 22(2):179–214, 2004.

[7] J.J. Rocchio. Relevance feedback in information retrieval. In *The SMART Retrieval System: Experiments in Automatic Document Processing*, pages 313–323. Prentice-Hall, 1971.

[8] V. Lavrenko and W.B. Croft. Relevance-based language models. In *Proceedings of SIGIR*, pages 120–127, 2001.

[9] L. Gao, X. Ma, J. Lin, and J. Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of ACL*, pages 1762–1777, 2023.

[10] L. Wang, N. Yang, and F. Wei. Query2Doc: Query expansion with large language models. In *Proceedings of EMNLP*, pages 9414–9423, 2023.

[11] R. Jagerman, H. Zhuang, Z. Qin, X. Wang, and M. Bendersky. Query expansion by prompting large language models. *arXiv preprint arXiv:2305.03653*, 2023.

[12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of NeurIPS*, 2020.

[13] C. Macdonald, N. Tonellotto, S. MacAvaney, and I. Ounis. PyTerrier: Declarative experimentation in Python from BM25 to dense retrieval. In *Proceedings of CIKM*, pages 4526–4533, 2021.

[14] C.D. Manning, P. Raghavan, and H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.